MA550: Homework 2

Finite Differences and Immersed Boundary Methods

Axel Fraud
Colorado School of Mines
October 2026

Problem 1: Finite-Difference Solution of the Poisson Equation

Numerical Method

We consider the two-dimensional Poisson equation

$$\nabla^2 u(x, y) = f(x, y) \tag{1}$$

on the square domain

$$\Omega = [0, 2\pi] \times [0, 2\pi].$$

The Laplacian is approximated using the standard five-point finite-difference stencil,

$$\nabla^2 u(x_i, y_j) \approx \frac{u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j}}{\Delta x^2}. \tag{2}$$

This approximation is second-order accurate in the grid spacing,

$$\nabla_h^2 u = \nabla^2 u + \mathcal{O}(\Delta x^2).$$

The resulting sparse linear system is written as

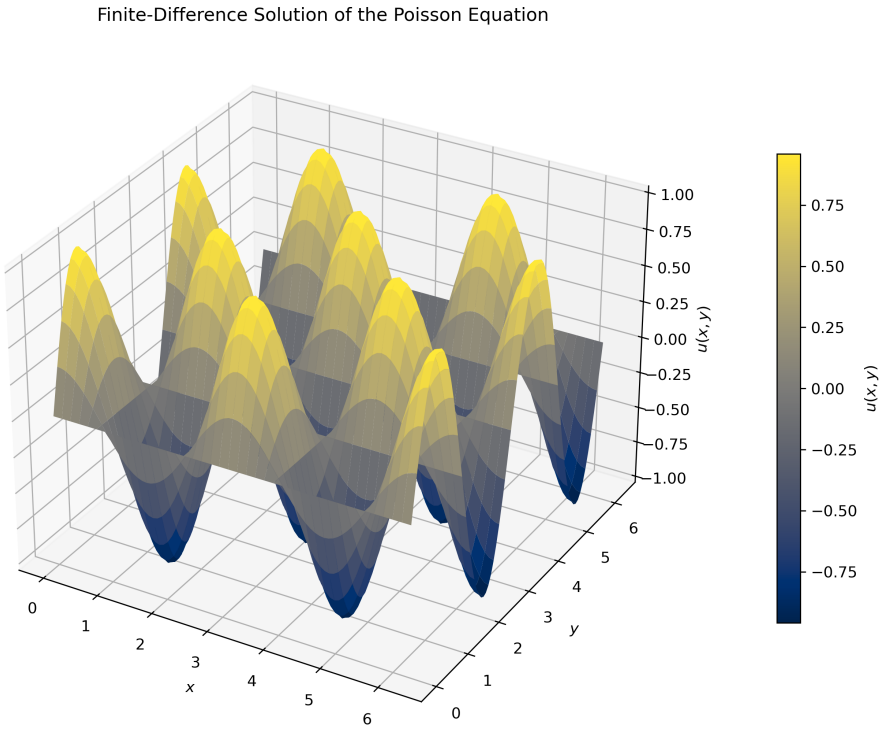
$$A\mathbf{u} = \mathbf{f}, \tag{3}$$

where A represents the discrete Laplacian.

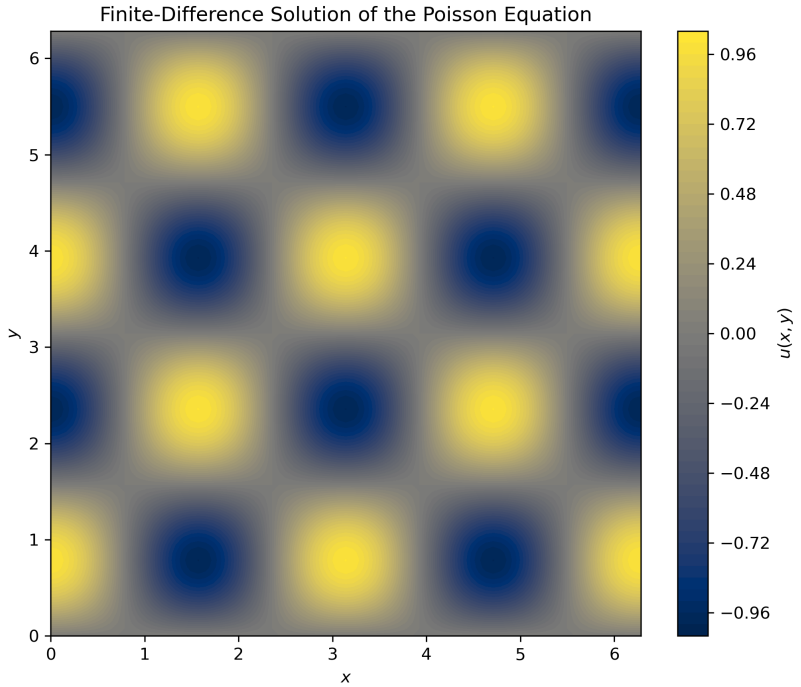
Numerical Solution

The numerical solution exhibits the expected oscillatory structure of the prescribed solution.

Figure 1 shows both the three-dimensional surface and its two-dimensional representation.



(a) Three-dimensional numerical solution.



(b) Two-dimensional contour representation.

Figure 1: Finite-difference solution of the Poisson equation.

Convergence Analysis

To evaluate numerical accuracy, the relative errors are computed using the discrete L_∞ and L_2 norms,

$$E_\infty = \frac{\|u_h - u_{\text{exact}}\|_\infty}{\|u_{\text{exact}}\|_\infty}, \tag{4}$$

and

$$E_2 = \frac{\|u_h - u_{\text{exact}}\|_2}{\|u_{\text{exact}}\|_2}. \tag{5}$$

The convergence results are shown in Figure 2.

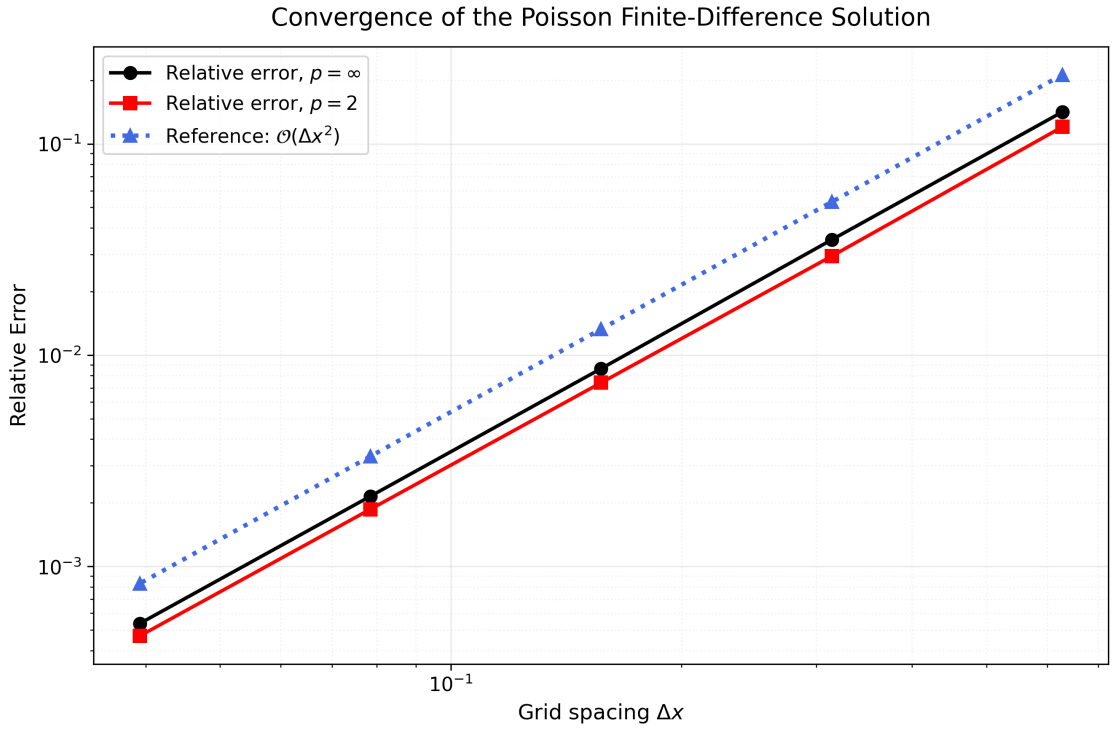


Figure 2: Relative numerical errors as functions of grid spacing. The dashed reference line represents second-order convergence, $\mathcal{O}(\Delta x^2)$.

Both error norms closely follow the second-order reference line. This confirms that the implementation achieves the expected convergence behavior of the five-point finite-difference approximation.

Problem 2: Poisson Equation with an Immersed Boundary

Mathematical Formulation

We extend the finite-difference Poisson solver by introducing an immersed boundary inside the computational domain.

The governing equation becomes

$$\nabla^2 u = f + S^{ib}(q), \tag{6}$$

where S^{ib} is the immersed-boundary spreading operator and q represents the unknown boundary forcing.

The immersed-boundary constraint is imposed through the interpolation operator,

$$J^{ib}(u) = g. \tag{7}$$

The immersed boundary is parameterized by

$$x^{ib}(\theta) = \pi + \cos\theta, \tag{8}$$
$$y^{ib}(\theta) = \pi + \sin\theta, \tag{9}$$

where $\theta \in [0, 2\pi]$.

Spreading and Interpolation

The spreading operator transfers forces defined on the immersed boundary to the Cartesian grid.

In continuous form,

$$S^{ib}(q)(x, y) = \int_\Gamma q(s)\, \delta_a(x - X(s))\delta_a(y - Y(s))\, ds. \tag{10}$$

The interpolation operator evaluates the Cartesian solution at the immersed-boundary points,

$$J^{ib}(u)(s) = \int_\Omega u(x, y)\delta_a(x - X(s))\delta_a(y - Y(s))\, dx\, dy. \tag{11}$$

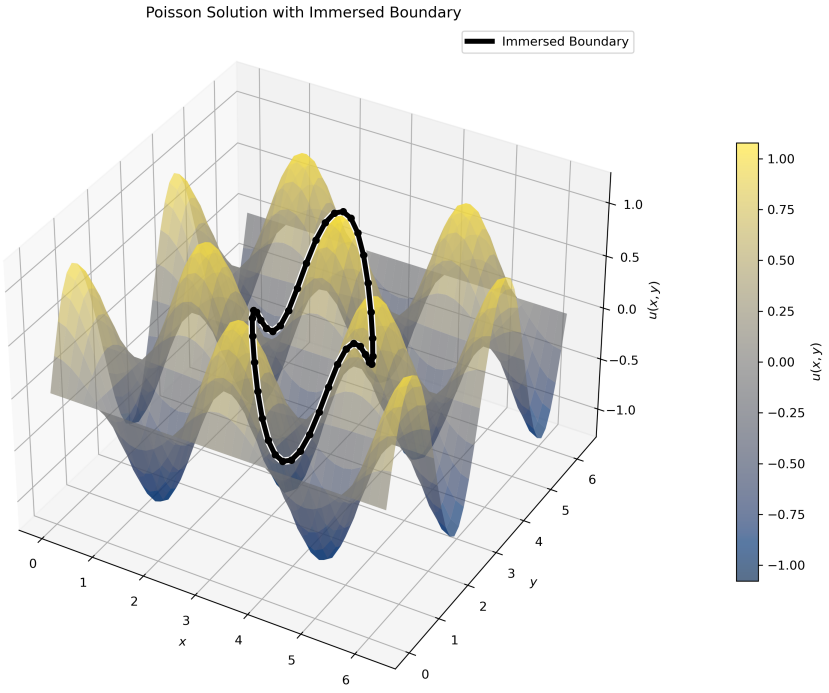
Here, δ_a denotes a regularized delta function.

After discretization, the coupled problem can be expressed as

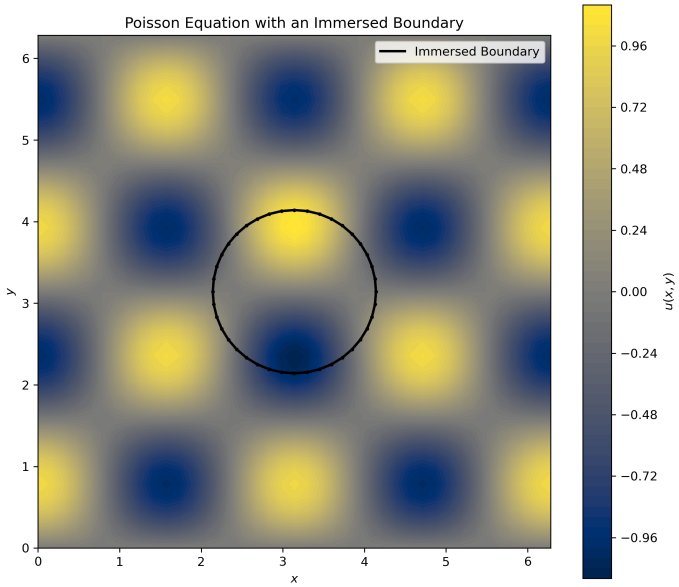
$$\begin{bmatrix} A & -S \\ J & 0 \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ \mathbf{q} \end{bmatrix} = \begin{bmatrix} \mathbf{f} \\ \mathbf{g} \end{bmatrix}. \tag{12}$$

Numerical Solution

Figure 3 illustrates the numerical solution with the immersed boundary superimposed.



(a) Three-dimensional solution.



(b) Two-dimensional solution with immersed boundary.

Figure 3: Numerical solution of the Poisson equation with an immersed boundary.

Grid Convergence

The convergence study is shown in Figure 4.

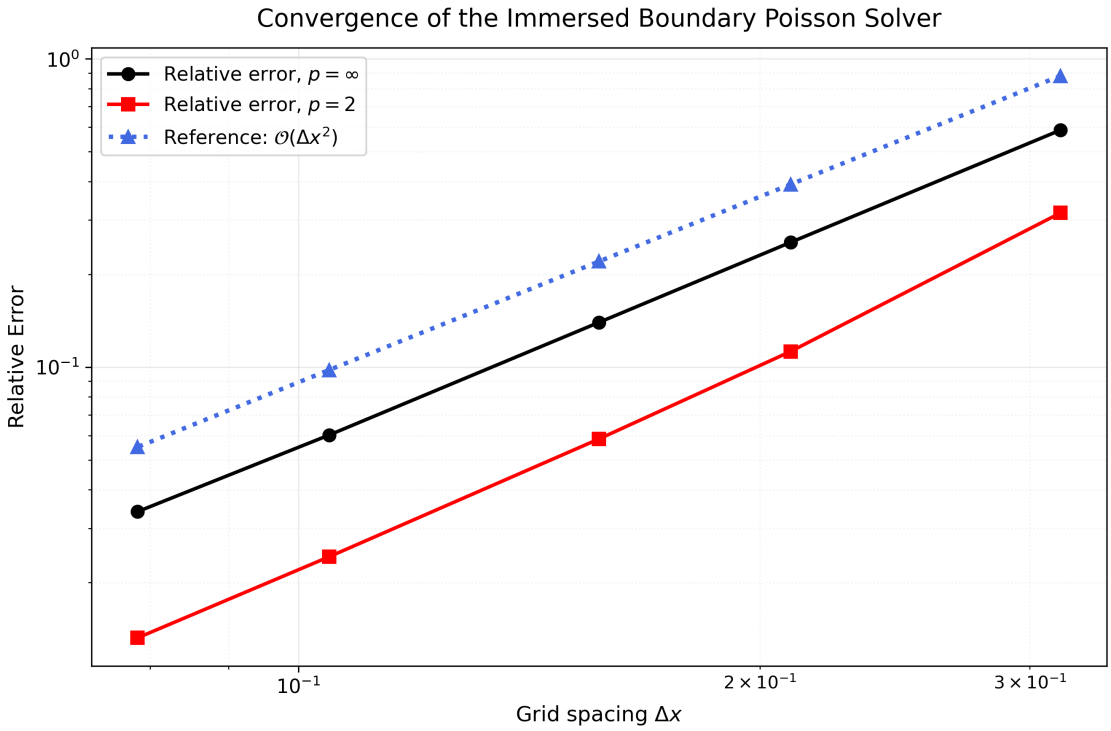


Figure 4: Relative errors for the immersed-boundary Poisson solver. The dashed line indicates second-order convergence.

The numerical errors decrease as the Cartesian grid is refined. The L_∞ error closely follows the second-order reference line, while the L_2 error also decreases consistently.

Effect of Immersed-Boundary Spacing

The spacing between immersed-boundary points influences both numerical accuracy and computational cost.

Figures 5 and 6 illustrate these effects.

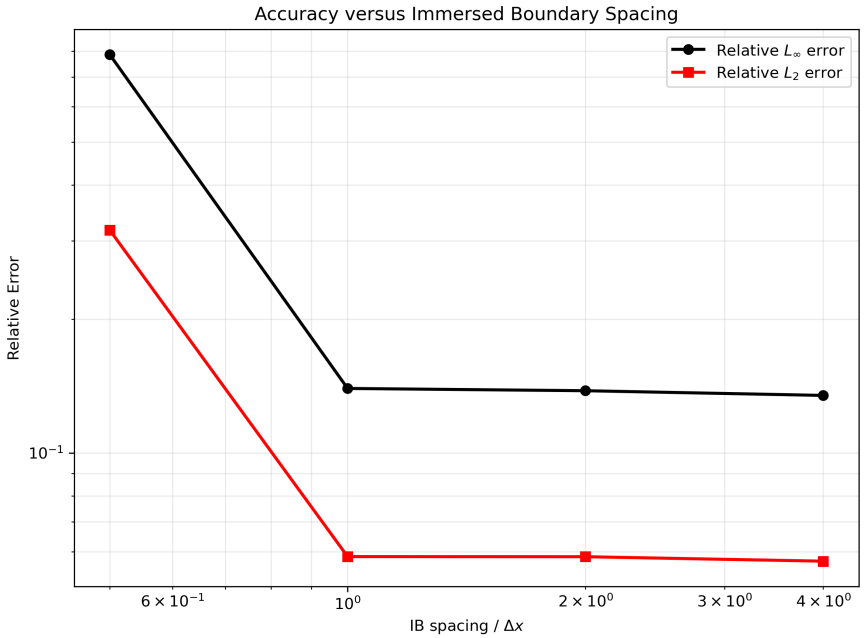


Figure 5: Relative error as a function of immersed-boundary spacing normalized by Cartesian grid spacing.

The errors are largest when the immersed-boundary points are spaced substantially closer than the Cartesian grid spacing. Increasing the spacing initially produces a significant improvement, after which the errors approximately stabilize.

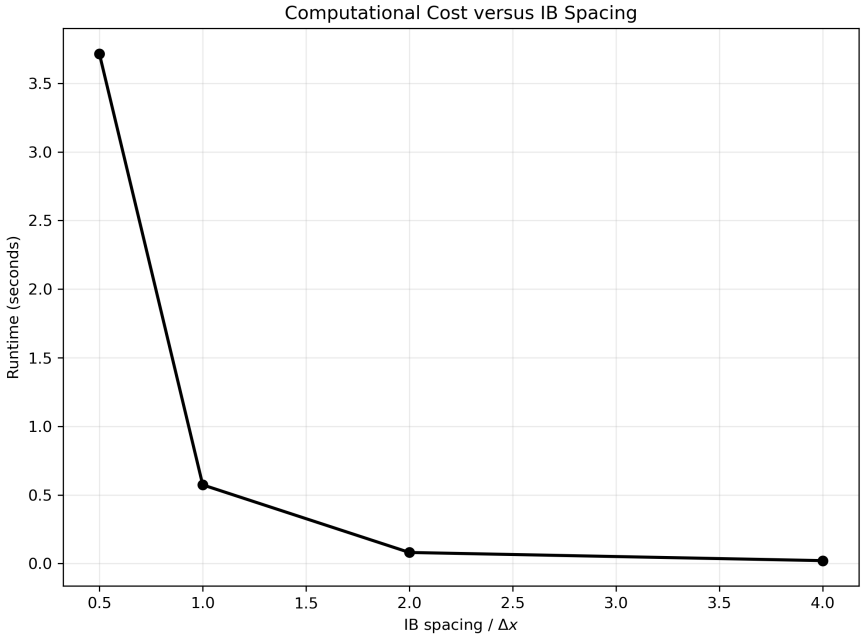


Figure 6: Computational runtime as a function of immersed-boundary spacing.

The computational cost decreases considerably as the immersed-boundary spacing increases.

These results illustrate the trade-off between immersed-boundary resolution and computational efficiency. For this experiment, spacing comparable to the Cartesian grid spacing provides a reasonable balance.

Problem 3: An Almost Minimal Surface

Problem Formulation

We now consider the homogeneous Poisson equation

$$\nabla^2 u = S^{ib}(q), \tag{13}$$

with homogeneous Dirichlet conditions on the outer boundary,

$$u(0, y) = u(2\pi, y) = u(x, 0) = u(x, 2\pi) = 0. \tag{14}$$

The immersed boundary is parameterized by

$$x^{ib}(\theta) = \pi + \cos\theta, \tag{15}$$
$$y^{ib}(\theta) = \pi + \sin\theta, \tag{16}$$

and the prescribed values are

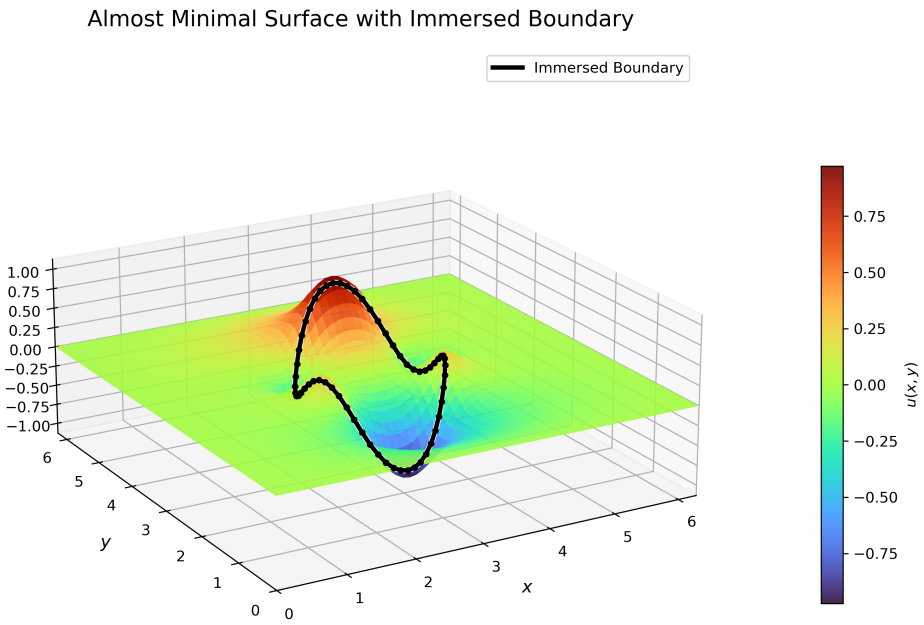
$$u(x^{ib}, y^{ib}) = \cos(2x^{ib})\sin(2y^{ib}). \tag{17}$$

Unlike the previous problem, the forcing term is zero throughout the domain except for the contribution from the immersed boundary.

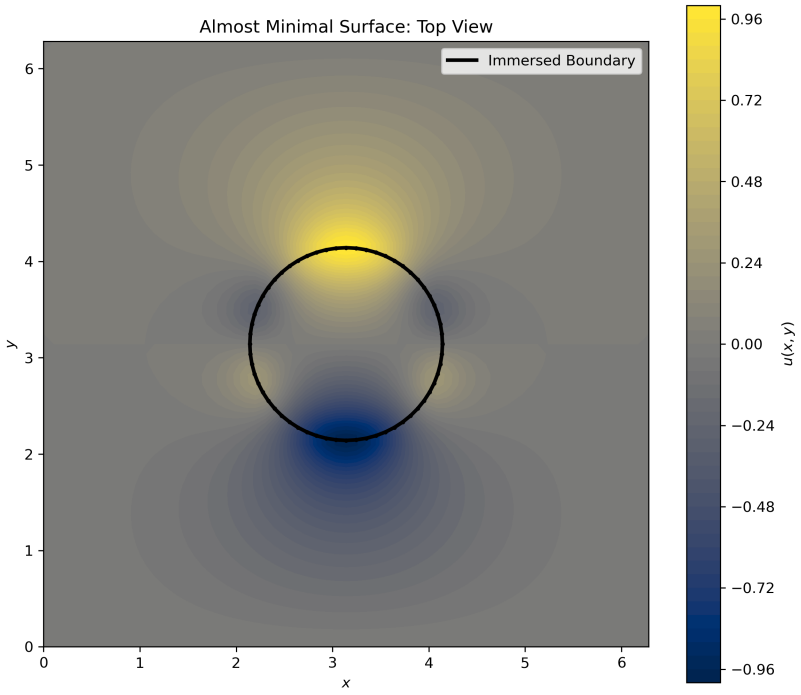
The resulting solution is therefore determined by the prescribed values along the circular boundary and the homogeneous conditions on the outer boundary.

Numerical Results

The computed solution is shown in Figure 7.



(a) Three-dimensional almost minimal surface.



(b) Two-dimensional contour representation.

Figure 7: Numerical solution of the homogeneous Poisson equation with prescribed immersed-boundary values.

The surface exhibits positive and negative deformations near the immersed boundary, corresponding to the oscillatory prescribed boundary values.

Away from the immersed boundary, the solution gradually approaches zero as required by the homogeneous outer Dirichlet conditions.

The resulting surface resembles a minimal surface in the sense that it smoothly connects the prescribed boundary values. However, it is not a true minimal surface because the governing equation is linear, whereas the minimal-surface equation is nonlinear.

Problem 4: Charlie's Rule

For this problem, I explored several extensions of the immersed boundary method developed in Problem 3. In particular, I investigated how multiple moving immersed boundaries influence the numerical solution and whether reusing previous solutions as initial guesses could improve the efficiency of GMRES.

The goal was to extend the original static immersed boundary problem into a pseudo-dynamic simulation while examining both the resulting surface geometries and computational performance.

Moving Immersed Boundaries

Rather than considering a single stationary immersed boundary, I introduced two immersed boundaries whose positions and geometries vary throughout the simulation.

The first boundary is an ellipse that undergoes rotation, while the second is a circle that undergoes periodic deformation.

At each frame, the immersed boundary locations are updated, and the corresponding numerical solution is recomputed.

This produces a pseudo-dynamic simulation in which the geometry changes over time, but the governing elliptic problem is solved independently at each configuration.

Mathematically, the numerical solution at frame k satisfies

$$\mathcal{L}_h u^{(k)} + \mathcal{S}^{(k)} F^{(k)} = f, \tag{18}$$

where $\mathcal{L}_h$ is the discrete Laplacian, $\mathcal{S}^{(k)}$ is the spreading operator associated with the immersed boundary geometry at frame k, and $F^{(k)}$ represents the immersed boundary forces.

The dependence on k reflects the changing geometry of the immersed boundaries.

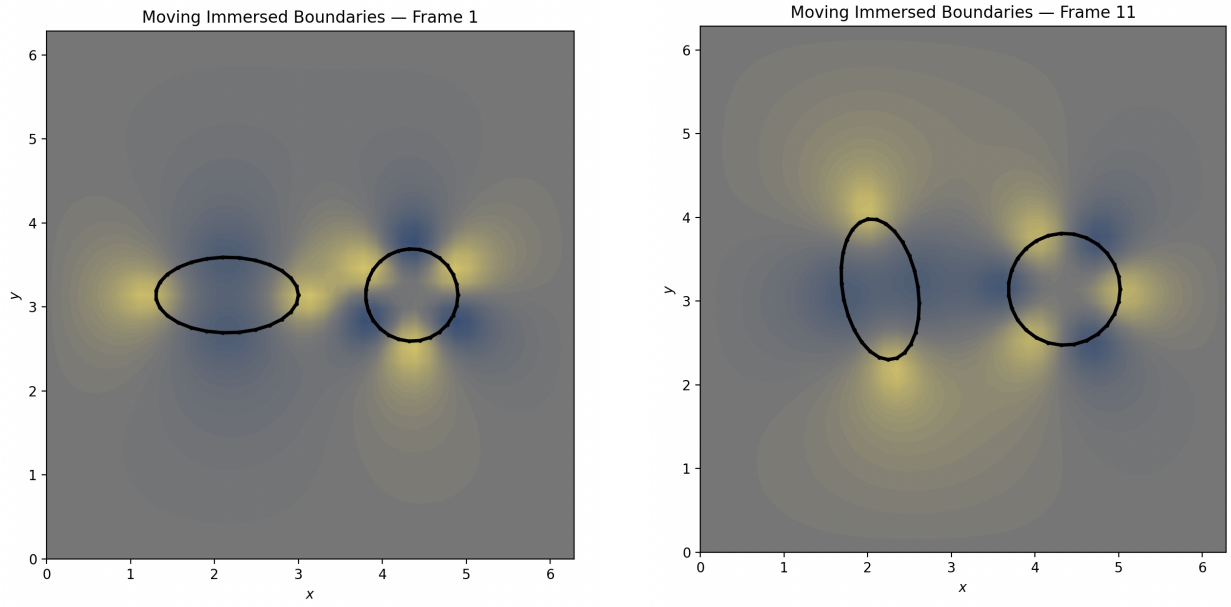
Two-Dimensional Visualization

Figure 8 shows two representative frames from the two-dimensional simulation.

The black curves represent the immersed boundaries, while the background contour plot illustrates the numerical solution $u(x, y)$.

As the boundaries move and deform, the surrounding solution changes accordingly.

In particular, the interaction between the two immersed boundaries produces increasingly complex spatial structures that would not appear in the single-boundary configuration.



(a) Initial configuration.

(b) Later configuration.

Figure 8: Two-dimensional snapshots of the moving immersed boundary simulation. The ellipse rotates while the circular boundary deforms, producing changes in the numerical solution.

Three-Dimensional Surface Deformation

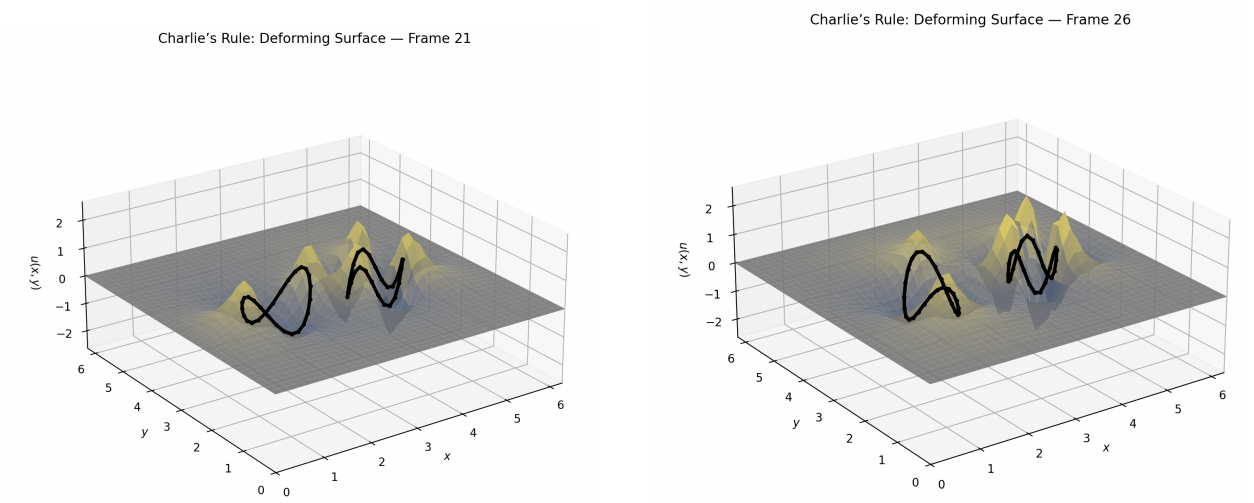
To better visualize the influence of the immersed boundaries, I also generated three-dimensional representations of the numerical solution.

Figure 9 shows two representative frames from the resulting animation.

The surface height corresponds to the numerical solution $u(x, y)$, while the black curves indicate the immersed boundaries embedded within the surface.

As the boundaries evolve, the resulting surface develops changing peaks, valleys, and localized deformations.

These visualizations demonstrate how relatively simple geometric motions can generate complicated surface structures through the immersed boundary method.



(a) First surface configuration.

(b) Later surface configuration.

Figure 9: Three-dimensional snapshots illustrating the deformation of the numerical solution as the immersed boundaries move. The changing boundary configurations produce different localized surface features.

GMRES Performance: Warm versus Cold Starts

An additional experiment investigated whether reusing the solution from the previous frame could improve the convergence of GMRES.

In the original implementation, GMRES begins with a zero initial guess,

$$u_0^{(k)} = 0. \tag{19}$$

This is referred to as a *cold start*.

However, because the immersed boundaries move continuously between frames, the numerical solutions at consecutive frames are expected to be similar.

This suggests using the previous solution as the initial guess for the next solve,

$$u_0^{(k)} = u^{(k-1)}, \tag{20}$$

which is referred to as a *warm start*.

The motivation is that if

$$\left\| u^{(k)} - u^{(k-1)} \right\| \ll \left\| u^{(k)} \right\|, \tag{21}$$

then the previous solution should provide a better approximation to the new solution than the zero vector.

In principle, this could reduce the number of Krylov iterations required for convergence.

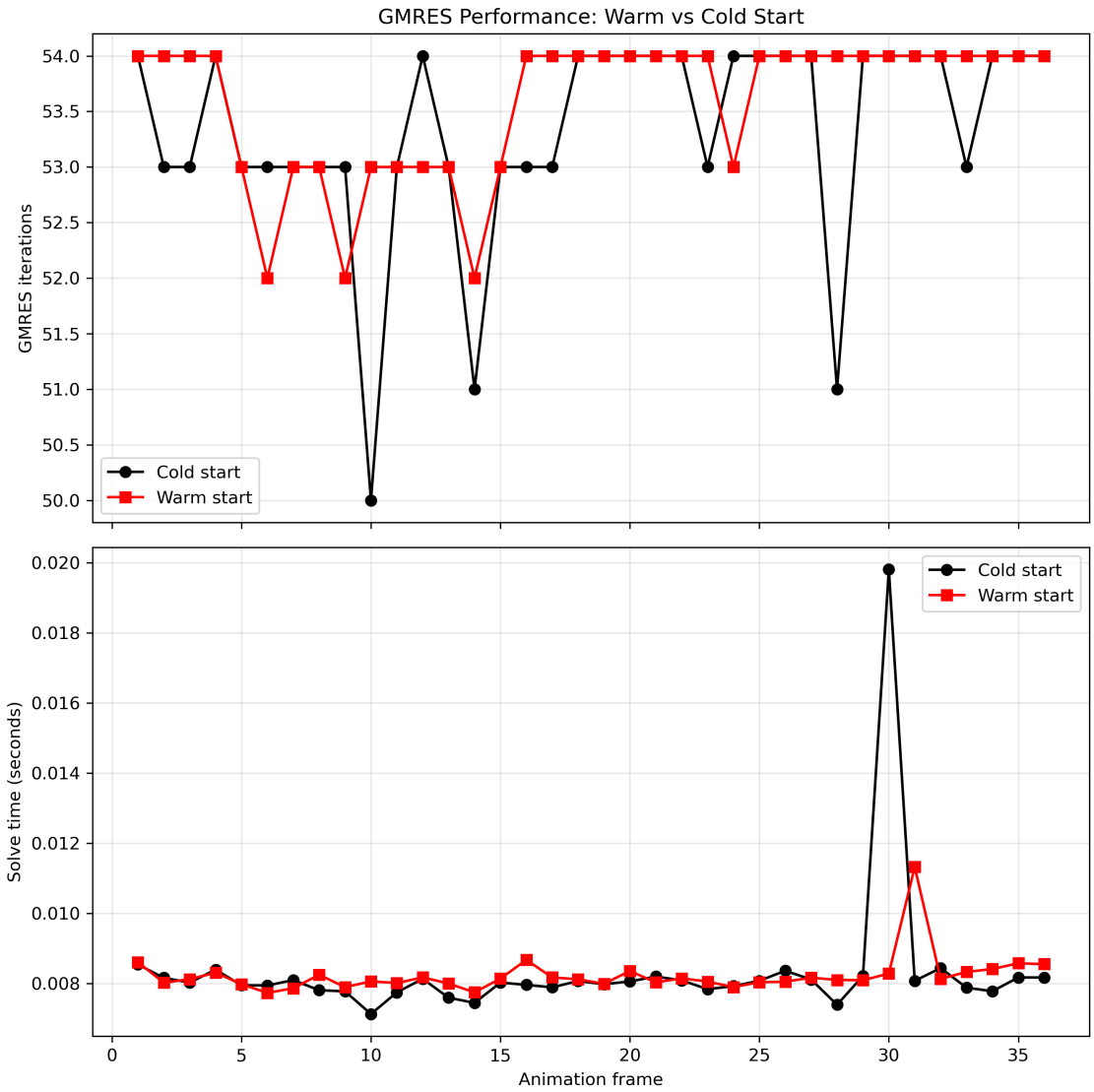


Figure 10: Comparison of GMRES performance using cold and warm starts over successive animation frames. The upper panel shows the number of GMRES iterations required for convergence, while the lower panel compares the corresponding solution times.

Discussion of GMRES Results

Interestingly, the numerical results indicate that warm-starting GMRES does not provide a significant computational advantage for this particular experiment.

As shown in Figure 10, both methods require approximately 50–54 iterations across the simulation.

Furthermore, the solution times remain comparable, with the warm-start approach occasionally requiring slightly more time than the cold-start approach.

Although the previous solution provides a physically reasonable initial guess, this does not necessarily guarantee faster convergence of the Krylov subspace method.

GMRES convergence depends on several factors, including the structure of the linear system, the initial residual, and the spectral properties of the operator.

Consequently, a better initial approximation does not always produce a substantial reduction in the number of iterations.

These results suggest that for this particular system, improving the preconditioning strategy may be more effective than simply reusing the previous solution.

Conclusion

This experiment demonstrates how the immersed boundary method can be extended beyond a single stationary boundary to simulate multiple moving and deforming geometries.

The resulting two-dimensional and three-dimensional visualizations illustrate how changes in boundary geometry influence the numerical solution.

Additionally, the GMRES comparison provides an interesting computational observation: although warm-starting is a natural strategy for a sequence of related linear systems, it does not necessarily improve convergence or computational efficiency.

Overall, these experiments highlight both the flexibility of the immersed boundary method and the importance of testing numerical optimization strategies rather than assuming they will improve performance.